DOCUMENTATION 1

#ELU

```
def ELU(x, n, a=0.01):
    """
    ELU(n, a=0.01)

    Applies the Exponential Linear Unit (ELU) activation function with a
custom threshold.
    For inputs greater than the threshold `n`, the function returns the input
unchanged.
    For inputs less than or equal to `n`, it applies an exponential
transformation:

    ELU(x, n, a) = x, if x > n
                   a * (exp(x) - 1), if x <= n

    This variant allows shifting the activation threshold, making it useful
for
    scenarios where the standard ELU (typically with n=0) needs adjustment.

    Args:
        x (float or torch.Tensor): Input value to activate.
        n (float): Threshold value that separates the linear and exponential
regimes.
        a (float, optional): Scale factor for the exponential part. Controls
the
            saturation value for negative/left-side inputs. Default: 0.01.

    Returns:
        torch.Tensor: A 1-element tensor containing the activated output.

    Note:
        - The function currently returns a single-element tensor. For batch
processing,
          consider using torch.where() for vectorization.
        - For x <= n, the output approaches -a as x → -∞, similar to standard
ELU.

    Examples:
        >>> from Jumping_LLM_Flash.scr.architecture import ELU
        >>> import torch
        >>>
```

```
        >>> # Example 1: Input greater than threshold
        >>> x = torch.tensor(2.0)
        >>> output = ELU(x, n=0.0, a=0.01)
        >>> print(output)  # tensor([2.0000])
        >>>
        >>> # Example 2: Input less than threshold
        >>> x = torch.tensor(-1.0)
        >>> output = ELU(x, n=0.0, a=0.01)
        >>> print(output)  # tensor([-0.0063])
        >>>
        >>> # Example 3: Custom threshold
        >>> x = torch.tensor(0.5)
        >>> output = ELU(x, n=1.0, a=0.1)
        >>> print(output)  # tensor([0.0649]) since 0.5 <= 1.0
    """
    if x > n:
        return x
    elif x <= n:
        return torch.tensor([a * (math.exp(x) - 1)])
```

Return type: `torch.Tensor`

Feed_Forward_Network

`class Feed_Forward_Network(dim, hidden_dim, activation, dropout=None)`

Description

A feed-forward neural network block with optional gating mechanism (SwiGLU-style).

This module implements a two-layer feed-forward network where the first linear layer projects the input to double the hidden dimension, applies an activation function, and then projects back to the original dimension. The doubling of the hidden dimension is designed to support gated linear units (like SwiGLU) where the output is split into two parts for gating.

Parameters

Parameter	Type	Description
`dim`	`int`	Input and output feature dimension.
`hidden_dim`	`int`	Hidden dimension size (the actual hidden dimension will be `hidden_dim * 2` before activation).
`activation`	`nn.Module`	Activation function to apply after the first linear layer. Examples: `nn.GELU()`, `nn.ReLU()`, `nn.SiLU()` for SwiGLU.

Parameter	Type	Description
`dropout`	`float`	Optional. Dropout probability. If provided and not `None`, a Dropout layer is inserted between the activation and the final linear layer. Default: `None` (no dropout).

Shape

- **Input:** `(batch_size, dim)`
- **Output:** `(batch_size, dim)`

Returns

`torch.Tensor` – Output tensor of shape `(batch_size, dim)`

Notes

- The first linear layer projects to `hidden_dim * 2` to support splitting into two parts for SwiGLU-style gating, but the actual split operation should be handled inside the activation function (e.g., custom SwiGLU).
- For standard activation functions without gating, the dimension doubling may be unnecessary; consider using `hidden_dim` directly in that case.

Examples

```
from Jumping_LLM_Flash.scr.architecture import Feed_Forward_Network
import torch.nn as nn

# Example with GELU activation and dropout
ffn = Feed_Forward_Network(
    dim=512,
    hidden_dim=2048,
    activation=nn.GELU(),
    dropout=0.1
)

# Example without dropout
ffn = Feed_Forward_Network(
    dim=768,
    hidden_dim=3072,
    activation=nn.SiLU()  # For SwiGLU
)

# Forward pass
```

```
x = torch.randn(2, 512)
output = ffn(x)
print(output.shape)  # torch.Size([2, 512])
```

GMQA_with_KV

`class GMQA_with_KV(d_model, num_heads, num_kv_groups)`

Description

Grouped Multi-Query Attention with Key-Value caching support.

This module implements Grouped Multi-Query Attention (GMQA), a variant of multi-head attention where the Key and Value projections are shared across groups of attention heads, reducing memory and computational costs while maintaining performance. Unlike standard Multi-Query Attention (MQA) which uses a single KV head, GMQA uses `num_kv_groups` groups, providing a flexible trade-off between efficiency and representational power.

Attention Mechanism

1. Query is projected to `num_heads` heads (standard multi-head)
2. Key and Value are projected to `num_kv_groups` heads
3. KV is repeated `num_heads // num_kv_groups` times to match query heads
4. Standard scaled dot-product attention is computed
5. Outputs are concatenated and projected back

Parameters

Parameter	Type	Description
`d_model`	`int`	Model dimension. Must be divisible by `num_heads`.
`num_heads`	`int`	Total number of attention heads.
`num_kv_groups`	`int`	Number of groups for Key/Value heads. Must divide `num_heads`.

Raises

- `AssertionError`: If `d_model` is not divisible by `num_heads` or if `num_heads` is not divisible by `num_kv_groups`.

Shape

- **Input:** `(batch_size, seq_len, d_model)`

- **Output:** `(batch_size, seq_len, d_model)`
- **Attention weights:** `(batch_size, num_heads, seq_len, seq_len)`
- **Present KV:** Tuple of `(K, V)` each of shape `(batch_size, num_kv_groups, seq_len, d_k)`

Returns

`Tuple[torch.Tensor, torch.Tensor, Optional[Tuple[torch.Tensor, torch.Tensor]]]`:

- Output tensor of shape `(batch_size, seq_len, d_model)`
- Attention weights of shape `(batch_size, num_heads, seq_len, seq_len)`
- Cached `(K, V)` tuple if `use_cache=True`, else `None`

Attributes

Attribute	Type	Description
`cached_k`	`torch.Tensor`	Buffer for cached Key tensors.
`cached_v`	`torch.Tensor`	Buffer for cached Value tensors.
`d_k`	`int`	Dimension per head (`d_model // num_heads`).

Notes

- GMQA offers a middle ground between standard Multi-Head Attention (MHA) and Multi-Query Attention (MQA)
- Memory efficiency increases as `num_kv_groups` decreases
- For `num_kv_groups == 1`, this becomes standard MQA
- For `num_kv_groups == num_heads`, this becomes standard MHA

Examples

```
from Jumping_LLM_Flash.scr.architecture import GMQA_with_KV

# Initialize with 8 heads and 2 KV groups
gmqa = GMQA_with_KV(d_model=512, num_heads=8, num_kv_groups=2)

# Standard forward pass
x = torch.randn(2, 10, 512)  # (batch, seq_len, d_model)
output, attn_weights, _ = gmqa(x)
print(output.shape)  # torch.Size([2, 10, 512])
print(attn_weights.shape)  # torch.Size([2, 8, 10, 10])
```

```python
# With KV caching for autoregressive generation
x1 = torch.randn(2, 1, 512)  # First token
x2 = torch.randn(2, 1, 512)  # Second token

output1, _, present_kv = gmqa(x1, use_cache=True)
output2, _, _ = gmqa(x2, use_cache=True, past_kv=present_kv)

# KV group dimension calculation
kv_dim = gmqa.d_k * gmqa.num_kv_groups
print(f"KV projection dimension: {kv_dim}")  # 64 (8 * 8)
```

GQA

`class GQA(d_model, num_heads, num_kv_heads=None, pos_enc=None)`

Description

Grouped Query Attention (GQA) with optional positional encoding.

This module implements Grouped Query Attention (GQA), an efficient variant of multi-head attention where Key and Value projections use fewer heads than Query projections. Each KV head is shared among multiple query heads, reducing memory footprint and computational cost while maintaining better performance than Multi-Query Attention (MQA).

Attention Mechanism

1. Query is projected to `num_heads` heads (standard multi-head)
2. Key and Value are projected to `num_kv_heads` heads (reduced count)
3. KV heads are repeated `num_heads // num_kv_heads` times to match query heads
4. Optional positional encoding is applied to Q and K
5. Standard scaled dot-product attention is computed
6. Outputs are concatenated and projected back

Parameters

Parameter	Type	Description
`d_model`	`int`	Model dimension. Must be divisible by `num_heads`.
`num_heads`	`int`	Total number of query attention heads.
`num_kv_heads`	`int`	Optional. Number of Key/Value heads. If `None`, defaults to `num_heads` (standard multi-head attention). Must divide `num_heads`.

Parameter	Type	Description
`pos_enc`	`Optional[Callable]`	Optional. Positional encoding function to apply to Q and K projections. Can be any callable that takes a tensor and returns a tensor of the same shape. Default: `None`.

Raises

- `AssertionError`: If `d_model` is not divisible by `num_heads`.

Shape

- **Input:** `(batch_size, seq_len, d_model)`
- **Output:** `(batch_size, seq_len, d_model)`

Returns

`torch.Tensor` – Output tensor of shape `(batch_size, seq_len, d_model)`

Attributes

Attribute	Type	Description
`d_k`	`int`	Dimension per head (`d_model // num_heads`).
`num_kv_heads`	`int`	Number of KV heads (defaults to `num_heads`).

Notes

- When `num_kv_heads == num_heads`, this becomes standard Multi-Head Attention (MHA)
- When `num_kv_heads == 1`, this becomes Multi-Query Attention (MQA)
- GQA offers a balanced trade-off between MHA (best quality, high memory) and MQA (lower quality, efficient)
- For autoregressive generation, GQA significantly reduces KV cache size compared to MHA
- The repeat operation uses `repeat_interleave` which is memory efficient

Examples

```
from Jumping_LLM_Flash.scr.architecture import GQA

# Example 1: Standard GQA with 4 query heads and 2 KV heads
model = GQA(d_model=100, num_heads=4, num_kv_heads=2)
```

```python
x = torch.randn(32, 10, 100)  # (batch, seq_len, d_model)
out = model(x)
print(out.shape)  # torch.Size([32, 10, 100])

# Example 2: GQA with positional encoding
class SinusoidalPosEnc(nn.Module):
    def forward(self, x):
        # Simplified positional encoding
        return x + torch.sin(torch.arange(x.size(1))).to(x.device)

model = GQA(
    d_model=512,
    num_heads=8,
    num_kv_heads=4,
    pos_enc=SinusoidalPosEnc()
)

# Example 3: Standard MHA (num_kv_heads defaults to num_heads)
model = GQA(d_model=512, num_heads=8)  # Same as multi-head attention

# Example 4: Multi-Query Attention (single KV head)
model = GQA(d_model=512, num_heads=8, num_kv_heads=1)

# Example 5: Different KV group configurations
configs = [
    (8, 8),   # MHA - 8 groups
    (8, 4),   # GQA - 4 groups
    (8, 2),   # GQA - 2 groups
    (8, 1),   # MQA - 1 group
]

for num_heads, num_kv_heads in configs:
    model = GQA(d_model=512, num_heads=num_heads, num_kv_heads=num_kv_heads)
    print(f"Heads: {num_heads}, KV heads: {num_kv_heads}, Ratio:
{num_heads//num_kv_heads}x")
```

leaky_ReLU

`leaky_ReLU(x, n, a=0.01)`

Description

Applies the Leaky Rectified Linear Unit (Leaky ReLU) activation function with a custom threshold.
For inputs greater than or equal to the threshold `n`, the function returns the input unchanged.
For inputs less than `n`, it returns the input scaled by a small factor `a` (typically 0.01), allowing a small gradient for negative values and preventing dead neurons.
This variant allows shifting the activation threshold, making it useful for scenarios where the standard Leaky ReLU (typically with `n=0`) needs adjustment.

```
from Jumping_LLM_Flash.scr.architecture import leaky_ReLU


print(leaky_ReLU(2.0, 0.0))
print(leaky_ReLU(-1.0, 0.0))


print(leaky_ReLU(0.5, 1.0, 0.1))
print(leaky_ReLU(1.5, 1.0, 0.1))


print(leaky_ReLU(5, 3))
print(leaky_ReLU(2, 3))
```

MoE

`class MoE(expert, input_dim, num_experts)`

Description

Mixture of Experts (MoE) module with learnable gating mechanism.

This module implements a Mixture of Experts architecture where multiple expert networks process the input in parallel, and a learnable gating network determines the contribution of each expert to the final output. The gate produces a softmax distribution over experts, and the final output is a weighted sum of all expert outputs.

How It Works

1. Input is passed through a linear gating network to produce logits
2. Softmax is applied to obtain expert weights (probabilities)
3. Each expert processes the input independently
4. Expert outputs are combined using the gate weights as coefficients

Parameters

Parameter	Type	Description
`expert`	`nn.ModuleList` or `list` of `nn.Module`	List of expert networks. Each expert must accept the same input shape and produce outputs of consistent dimensions.
`input_dim`	`int`	Input feature dimension. Used for the gating network.
`num_experts`	`int`	Number of experts. Must match the length of `expert`.

Shape

- **Input:** `(batch_size, *input_shape)` where `*input_shape` can be arbitrary
- **Gate weights:** `(batch_size, num_experts)`
- **Output:** Same shape as individual expert output `(batch_size, output_dim)`

Returns

`torch.Tensor` – Weighted combination of expert outputs.

Notes

- The gating network flattens the input to a vector of size `(batch_size, input_dim)` before projection, so input can be multi-dimensional.
- All experts are applied to every input (dense MoE), not sparse routing.
- For sparse MoE with top-k routing, consider modifying the gate to select only the top-k experts for each input.
- The module uses standard softmax, so gate weights sum to 1 for each sample.
- Training multiple experts simultaneously can be computationally expensive.
- Gradient flow goes through both the gate and all experts.

Examples

```
from Jumping_LLM_Flash.scr.architecture import MoE


expert1 = nn.Sequential(nn.Linear(10, 20), nn.ReLU(), nn.Linear(20, 5))
expert2 = nn.Sequential(nn.Linear(10, 20), nn.Tanh(), nn.Linear(20, 5))
expert3 = nn.Sequential(nn.Linear(10, 20), nn.GELU(), nn.Linear(20, 5))
experts = nn.ModuleList([expert1, expert2, expert3])
```

```
moe = MoE(expert=experts, input_dim=10, num_experts=3)

x = torch.randn(4, 10)  # (batch_size, input_dim)
output = moe(x)
print(output.shape)  # torch.Size([4, 5])
```

MQA

`class MQA(d_model, num_heads)`

Description

Multi-Query Attention (MQA) module with shared Key and Value projections.

This module implements Multi-Query Attention (MQA), an efficient variant of multi-head attention where all attention heads share a single Key and Value projection. Unlike standard Multi-Head Attention (MHA) which has separate K and V projections for each head, MQA uses a single K and V projection that is broadcasted across all query heads.

Attention Mechanism

1. Query is projected to `num_heads` heads (standard multi-head)
2. Key and Value are projected to a single head (shared across all query heads)
3. The single K and V are broadcasted to match the number of query heads
4. Standard scaled dot-product attention is computed
5. Outputs are concatenated and projected back

This significantly reduces memory usage and computational cost, especially for autoregressive generation where KV cache size is reduced by a factor of `num_heads`.

Parameters

Parameter	Type	Description
`d_model`	`int`	Model dimension. Must be divisible by `num_heads`.
`num_heads`	`int`	Number of attention heads.

Raises

- `AssertionError`: If `d_model` is not divisible by `num_heads`.

Shape

- **Input:** `(batch_size, seq_len, d_model)`
- **Output:** `(batch_size, seq_len, d_model)`
- **Attention scores:** `(batch_size, num_heads, seq_len, seq_len)`

Returns

`torch.Tensor` – Output tensor of shape `(batch_size, seq_len, d_model)`

Attributes

Attribute	Type	Description
`d_k`	`int`	Dimension per head (`d_model // num_heads`).
`num_heads`	`int`	Number of query attention heads.

Notes

- KV cache size is reduced by a factor of `num_heads` compared to MHA
- Memory efficient: K and V use only `d_model` parameters each (vs `d_model * num_heads` in MHA)
- Training can be faster but may slightly reduce model quality compared to MHA
- Used in models like PaLM, GPT-NeoX, and many efficient transformers
- For a middle ground between MHA and MQA, see `GQA` (Grouped Query Attention)

Examples

Basic Usage

```
from Jumping_LLM_Flash.scr.architecture import MQA

model = MQA(d_model=100, num_heads=4)
x = torch.randn(32, 10, 100)  # (batch, seq_len, d_model)
out = model(x)
print(out.shape)  # torch.Size([32, 10, 100])
```

RMSNorm

`class RMSNorm(dim, eps=1e-8)`

Description

Root Mean Square Layer Normalization (RMSNorm).
This module implements RMSNorm, a simplified version of Layer Normalization that uses only the root mean square of the inputs for normalization, omitting the mean centering step.
RMSNorm has been shown to be computationally more efficient while maintaining comparable performance to LayerNorm in many transformer architectures.

Mathematical Definition

RMSNorm(x) = scale * x / RMS(x)

where `RMS(x) = sqrt(mean(x^2)) = sqrt(sum(x^2) / dim)`

Comparison with LayerNorm

Aspect	RMSNorm	LayerNorm
Mean subtraction	No	Yes
Normalization	RMS only	Mean + Variance
Computational cost	Lower	Higher
Parameter count	`dim`	`dim * 2` (scale + bias)
Output mean	Not necessarily 0	Always ~0

Parameters

Parameter	Type	Description
`dim`	`int`	Feature dimension to normalize over (last dimension).
`eps`	`float`	Optional. Small epsilon value added to denominator for numerical stability. Default: `1e-8`.

Shape

- **Input:** `(*, dim)` where `*` can be any number of dimensions
- **Output:** Same shape as input `(*, dim)`

Returns

`torch.Tensor` – Normalized tensor of the same shape as input.

Attributes

Attribute	Type	Description
`scale`	`nn.Parameter`	Learnable scaling parameter of shape `(dim,)`.
`eps`	`float`	Epsilon for numerical stability.

Notes

- Normalizes along the last dimension (`dim=-1`)
- Uses root mean square instead of standard deviation
- Does not center the data (unlike LayerNorm)
- Recommended for transformer models, especially with larger hidden dimensions
- More stable during training due to fewer computations
- No bias term is used in RMSNorm (only scaling)
- Used in models like LLaMA, Mistral, and many modern LLMs

Examples

Basic Usage

```
from Jumping_LLM_Flash.scr.architecture import RMSNorm
rms_norm = RMSNorm(dim=512)
x = torch.randn(2, 10, 512)  # (batch, seq_len, dim)
output = rms_norm(x)
print(output.shape)  # torch.Size([2, 10, 512])
```

RoPE

`RoPE(x)`

Description

Applies Rotary Positional Encoding (RoPE) to the input tensor.
Rotary Positional Encoding is a relative positional encoding method that incorporates position information by rotating the feature vectors in a high-dimensional space. Unlike absolute positional encodings (like in original Transformer), RoPE encodes relative positions and has been shown to be more effective for long sequences.

Mathematical Definition

The encoding works by splitting the embedding dimension into pairs and applying a rotation matrix to each pair based on its position:
For a pair `(x1, x2)` at position `pos`:

$[x1'][cos(\theta) - sin(\theta)]x1$
$x2' = [sin(\theta)cos(\theta)]x2$

text

where `θ = pos / 10000^(2i/dim)` for each dimension pair `i`.

Key Properties

- **Relative Position Encoding**: Encodes relative positions, not absolute
- **Orthogonal Transformation**: Preserves vector norms (rotation)
- **Long-range Awareness**: Better performance on long sequences
- **No Learnable Parameters**: Deterministic encoding based on positions

Parameters

Parameter	Type	Description
`x`	`torch.Tensor`	Input tensor of shape `(batch_size, seq_len, dim)`. The last dimension must be even.

Returns

`torch.Tensor` – Tensor with Rotary Positional Encoding applied, same shape as input `(batch_size, seq_len, dim)`.

Raises

- `AssertionError`: If the last dimension (`dim`) is not even.

Shape

- **Input:** `(batch_size, seq_len, dim)` where `dim` is even
- **Output:** Same shape as input `(batch_size, seq_len, dim)`

Notes

- Requires the feature dimension to be even (for pairing)
- Uses the standard RoPE frequency formula: `1/(10000^(2i/dim))`
- Applied along the sequence dimension (relative position encoding)

- No learnable parameters, deterministic encoding
- Supports batched processing and GPU acceleration
- Preserves the norm of the input vectors (rotation is orthogonal)
- Can be applied to query and key projections separately in attention

Examples

Basic Usage

```
from Jumping_LLM_Flash.scr.architecture import RoPE
x = torch.randn(2, 10, 64)  # (batch, seq_len, dim)
x_rotated = RoPE(x)
print(x_rotated.shape)  # torch.Size([2, 10, 64])
```

SwigLU

`class SwigLU()`

Description

Swish-Gated Linear Unit (SwigLU) activation function.
This module implements the SwigLU (also known as SiLU-GLU or Swish-GLU) activation function, which combines the Swish (SiLU) activation with a gating mechanism. It takes an input tensor, splits it into two equal parts along the last dimension, applies Swish (`x * sigmoid(x)`) to the first part, and multiplies it element-wise with the second part.

Mathematical Definition

$SwigLU(x) = Swish(x1) * x2 = x1 * sigmoid(x1) * x2$

where `x1` and `x2` are the two halves of the input tensor split along the feature dimension.

Comparison with Other Gated Activations

Activation	Formula	Output Dim	Characteristics
GLU	`sigmoid(x1) * x2`	`dim/2`	Traditional gating
SwigLU	`swish(x1) * x2`	`dim/2`	Smooth, non-monotonic
GEGLU	`gelu(x1) * x2`	`dim/2`	Smooth, Gaussian-like
ReGLU	`relu(x1) * x2`	`dim/2`	Sparse, ReLU-based

Shape

- **Input:** `(*, dim)` where `dim` must be even
- **Output:** `(*, dim // 2)` (feature dimension is halved)

Returns

`torch.Tensor` – Activated tensor of shape `(*, dim // 2)`

Raises

- `RuntimeError` : If the last dimension is odd (cannot be split into two halves)

Notes

- The input dimension is halved in the output
- Uses SiLU (Swish) activation: `x * sigmoid(x)`
- No learnable parameters (deterministic)
- Commonly used in transformer FFN layers (e.g., LLaMA, PaLM)
- More computationally expensive than ReLU but often gives better quality
- Called "SwiGLU" in some papers (Swish-Gated Linear Unit)

Examples

Basic Usage

```
from Jumping_LLM_Flash.scr.architecture import SwigLU
swiglu = SwigLU()
x = torch.randn(2, 10, 512)  # (batch, seq_len, dim)
output = swiglu(x)
print(output.shape)  # torch.Size([2, 10, 256]) - dimension halved
```

positional_encodings

`positional_encodings(seq_len, d_model, device)`

Description

Generates sinusoidal positional encodings for transformer models.
This function creates the classic sinusoidal positional encodings introduced in the "Attention Is All You Need" paper. These encodings use sine and cosine functions of different frequencies to encode absolute positions in the sequence.

Mathematical Definition

For position `pos` and dimension `i`:

$PE(pos, 2i) = sin(pos/10000^(2i/d_model))$
$PE(pos, 2i+1) = cos(pos/10000^(2i/d_model))$

text

Key Properties

- **Deterministic**: No learnable parameters
- **Relative Position Awareness**: PE(pos + k) is a linear function of PE(pos)
- **Multi-frequency**: Different dimensions capture different time scales
- **Bounded Values**: All values are in range [-1, 1]
- **Orthogonal**: Different positions have low dot products

Parameters

Parameter	Type	Description
`seq_len`	`int`	Length of the sequence (number of positions to encode).
`d_model`	`int`	Model dimension (embedding size). Should be even for proper pairing of sine and cosine functions.
`device`	`torch.device`	Device to place the resulting tensor on (e.g., 'cpu', 'cuda', 'cuda:0').

Returns

`torch.Tensor` – Positional encodings tensor of shape `(seq_len, d_model)` with dtype `torch.float32`.

Shape

- **Output:** `(seq_len, d_model)`

Notes

- The encodings are deterministic and do not contain learnable parameters
- Sine is applied to even indices (0, 2, 4, ...), cosine to odd indices (1, 3, 5, ...)
- The frequency decreases with higher dimensions
- Works with any `d_model` (if odd, the last dimension uses only sine, no cosine pairing)

- Often added to input embeddings before the first transformer layer
- Supports both CPU and GPU tensors via the `device` parameter

Examples

Basic Usage

```
from Jumping_LLM_Flash.scr.architecture import positional_encodings
pe = positional_encodings(seq_len=10, d_model=512, device='cpu')
print(pe.shape)  # torch.Size([10, 512])
```

SentencePiece Tokenizers

SentencePieceBPE

`class SentencePieceBPE(vocab_size=10000, character_coverage=1.0)`

Description

SentencePiece-style Byte Pair Encoding (BPE) tokenizer.

This class implements a BPE tokenizer inspired by Google's SentencePiece, which treats the input text as a sequence of Unicode characters rather than words. It supports normalization, subword tokenization, and handles spaces as special characters (_). The tokenizer is designed for multilingual and raw text processing without requiring pre-tokenization.

Key Features

- **Raw Text Processing**: No need for pre-tokenization
- **Unicode Normalization**: NFKC normalization
- **Space Handling**: Special character (_) for spaces
- **BPE Subword Segmentation**: Iterative merging algorithm
- **Special Tokens**: `<unk>`, `<s>`, `</s>`, `<pad>`
- **Save/Load**: JSON serialization support

Parameters

Parameter	Type	Default	Description
`vocab_size`	`int`	`10000`	Target vocabulary size
`character_coverage`	`float`	`1.0`	Character coverage ratio (use 0.9995 for multilingual)

Attributes

Attribute	Type	Description
`vocab`	`dict`	Token → ID mapping
`reverse_vocab`	`dict`	ID → Token mapping
`merges`	`dict`	Token pair → merged token
`special_tokens`	`dict`	Special token → ID mapping

Examples

Basic Training and Usage

```python
from Jumping_LLM_Flash.scr.preprocessing import SentencePieceBPE

tokenizer = SentencePieceBPE(vocab_size=1000)

texts = [
    "Hello world!",
    "Machine learning is amazing.",
    "Natural language processing with SentencePiece."
]

tokenizer.train(texts, verbose=True)

# Encode text
pieces = tokenizer.encode_as_pieces("Hello world!")
print(pieces)  # ['_', 'H', 'e', 'll', 'o', '_', 'w', 'or', 'l', 'd', '!']

# Encode to IDs
ids = tokenizer.encode_as_ids("Hello world!")
print(ids)

# Decode back
decoded = tokenizer.decode_pieces(pieces)
print(decoded)  # "Hello world!"
```

BPE

`class BPE(vocab_size)`

Description

Byte Pair Encoding (BPE) tokenizer for text processing.

This class implements the Byte Pair Encoding algorithm for subword tokenization, originally introduced in "Neural Machine Translation of Rare Words with Subword Units" (Sennrich et al., 2015). BPE is a data compression technique adapted for NLP that iteratively merges the most frequent pairs of characters or subword units to build a vocabulary of subword tokens.

Key Features

- **Subword Tokenization**: Splits words into meaningful subword units
- **Vocabulary Learning**: Learns optimal subword units from corpus
- **Out-of-Vocabulary Handling**: Uses for unknown tokens
- **Special Tokens**: Supports BOS, EOS, PAD, UNK, and SPACE tokens
- **Batch Processing**: Efficient encoding of multiple texts
- **Padding Support**: Automatic padding to max sequence length
- **Bidirectional Encoding/Decoding**: Encode text to tokens and decode back

How BPE Works

1. Start with character-level tokens
2. Count frequencies of adjacent token pairs
3. Merge the most frequent pair
4. Repeat steps 2-3 until vocabulary size is reached
5. Use learned merges to tokenize new text

Parameters

Parameter	Type	Description
`vocab_size`	`int`	Target vocabulary size (number of merge operations to perform).

Attributes

Attribute	Type	Description
`vocab_size`	`int`	Target vocabulary size.
`bpe_codes`	`dict`	Dictionary mapping merged pairs to their merge order.
`bpe_vocab`	`dict`	Current vocabulary state during training.
`token_to_id`	`dict`	Mapping from tokens to integer IDs.

Attribute	Type	Description
`id_to_token`	`dict`	Mapping from integer IDs to tokens.
`unk_token`	`str`	Unknown token (default: '').
`pad_token`	`str`	Padding token (default: '').
`eos_token`	`str`	End-of-sequence token (default: '').
`bos_token`	`str`	Beginning-of-sequence token (default: '').
`space_token`	`str`	Space token (default: '').
`special_tokens`	`dict`	Dictionary of special tokens with their IDs.

Notes

- The tokenizer treats spaces as special tokens between words
- Unknown tokens are mapped to `<UNK>`
- Supports both word-level and character-level tokenization
- BPE merges are learned greedily based on frequency
- Vocabulary size includes special tokens
- Token IDs are assigned dynamically during training

Examples

Basic Training and Usage

```
from Jumping_LLM_Flash.scr.preprocessing import BPE

bpe = BPE(vocab_size=20)

# Train on example corpus
corpus = ["hello world", "hello there", "world of warcraft", "hello hello"]
bpe.fit(corpus)

# Encode text
tokens = bpe.encode("hello world")
print(tokens)  # ['he', 'll', 'o', '<SPACE>', 'wo', 'rl', 'd']

# Encode with special tokens
tokens_with_special = bpe.encode("hello world", add_special_tokens=True)
print(tokens_with_special)  # ['<BOS>', 'he', 'll', 'o', '<SPACE>', 'wo',
'rl', 'd', '<EOS>']

# Encode to IDs
```

```
ids = bpe.encode("hello world", return_ids=True)
print(ids)  # [5, 6, 7, 4, 8, 9, 10]

# Decode back to text
text = bpe.decode(tokens)
print(text)  # "hello world"
```

```
# SimpleTokenizer

`class SimpleTokenizer(text=None)`

## Description
A minimal character-level tokenizer for text processing.

This class implements a simple character-level tokenizer that maps individual
characters to integer IDs and vice versa. It is designed for educational
purposes and small-scale experiments where subword tokenization is not
necessary. The tokenizer creates a vocabulary from unique characters in the
training text and provides encoding/decoding functionality.

### Key Features
- **Character-level Tokenization**: Maps each character to an ID
- **Bidirectional Mapping**: Character ↔ ID conversion
- **Tensor Output**: PyTorch tensor integration
- **Minimal Dependencies**: Only requires torch
- **Lightweight**: Suitable for small vocabularies

## Parameters
| Parameter | Type | Default | Description |
|-----------|------|---------|-------------|
| `text` | `str` | `None` | Text to build vocabulary from. If provided,
vocabulary is built during initialization. |

## Attributes
| Attribute | Type | Description |
|-----------|------|-------------|
| `stoi` | `dict` | String-to-integer mapping (character → ID) |
| `itos` | `dict` | Integer-to-string mapping (ID → character) |
| `vocab_size` | `int` | Size of the vocabulary |

## Notes
- Character-level tokenization preserves all characters including spaces
- The vocabulary is built from unique characters only
```

- No special tokens (UNK, PAD, etc.) are included
- Designed for small datasets and educational purposes
- For production use, consider BPE or SentencePiece tokenizers

Examples

Initialize with Text
```python
from Jumping_LLM_Flash.scr.preprocessing import SimpleTokenizer

text = "hello world"
tokenizer = SimpleTokenizer(text)

print(tokenizer.vocab_size)  # Number of unique characters
print(tokenizer.stoi)  # {' ': 0, 'd': 1, 'e': 2, 'h': 3, 'l': 4, 'o': 5, 'r': 6, 'w': 7}
```

Quantization Utilities

Overview

This module provides utilities for model quantization and debugging, including:
- Activation capture via forward hooks
- Model information printing
- Layer-specific quantization
- Weight analysis and statistics

---

get_activation

`get_activation(name)`

Description
Creates a forward hook function to capture layer activations.

This function returns a hook that captures and stores activation information during the forward pass of a neural network layer. The captured data includes the activation output, layer weights, and bias (if available).

Parameters
| Parameter | Type | Description |

```
|-----------|------|-------------|
| `name` | `str` | Name identifier for the layer being monitored |

### Returns
`function` - A hook function that captures activation data

### Notes
- Stores activations in the global `activations` dictionary
- Captures output, weights, and bias tensors
- Tensors are detached to prevent gradient tracking
- Must be registered with `register_forward_hook`

### Examples
```python
from Jumping_LLM_Flash.scr.inference import get_activation

activations = {}

# Register hook
layer = nn.Linear(10, 5)
hook = layer.register_forward_hook(get_activation('my_layer'))

# Forward pass
x = torch.randn(2, 10)
output = layer(x)

# Access captured activations
print(activations['my_layer']['activation'].shape)
print(activations['my_layer']['weights'].shape)

# Remove hook
hook.remove()


## print_model_info

`print_model_info(model, x)`

### Description

Prints comprehensive information about a model's layers, parameters, and
activations.

This function performs a forward pass through the model while capturing and
displaying detailed information about each layer, including:
```
```

```
- Layer type and structure

- Parameter shapes and statistics

- Activation shapes and statistics

- Example computations for Linear layers


### Parameters

|Parameter|Type|Description|
|---|---|---|
|`model`|`nn.Module`|The neural network model to analyze|
|`x`|`torch.Tensor`|Input tensor for the forward pass|

### Returns

`torch.Tensor` - The output of the forward pass

### Notes

- Removes hooks after execution

- Shows detailed statistics for each layer

- Particularly useful for debugging and model inspection

- Works with Sequential and custom models


### Examples

python

from Jumping_LLM_Flash.scr.inference import print_model_info
model = nn.Sequential(
    nn.Linear(2, 4),
    nn.ReLU(),
    nn.Linear(4, 2)
)
x = torch.randn(1, 2)
output = print_model_info(model, x)
# Output will show:
# - Input shape
# - Each layer's parameters and activations
```

```
# - Statistics and example computations
# - Final output shape

### Sample Output

text

================================================================================
==
Input data: torch.Size([1, 2])
================================================================================
==
Layer 0 (Linear):
----------------------------------------
  weight: torch.Size([4, 2])
    Values: mean=-0.023456, std=0.123456, range=[-0.543210, 0.567890]
  bias: torch.Size([4])
    Values: mean=0.001234, std=0.001234, range=[-0.001234, 0.001234]
  Activation: torch.Size([1, 4])
    Statistics: mean=0.045678, std=0.234567, range=[-0.123456, 0.567890]
    Example computation (1 neuron):
      input * weights[0] + bias[0] = sum([0.1, -0.2] * [0.5, -0.3]...) + 0.001
= 0.1234
Final output: torch.Size([1, 2])
```

```
# Quantization Utilities for Language Models
## Overview
This module provides comprehensive quantization utilities for neural network
models, with special support for LLaMA-style language models. Key features
include:
- **Activation Capture**: Forward hooks for monitoring layer outputs
- **Model Inspection**: Detailed layer-by-layer information printing
- **8-bit Quantization**: Quantization of Linear layers
- **Error Analysis**: Precision loss measurement
- **Batch Support**: Handles models with tuple outputs
---
## get_activation_model
`get_activation_model(name)`
### Description
Creates a forward hook function for capturing activations from complex models
```

````
with tuple outputs.
### Parameters
| Parameter | Type | Description |
|-----------|------|-------------|
| `name` | `str` | Name identifier for the layer being monitored |
### Returns
`function` - A hook function that captures activation data
### Examples
```python
from Jumping_LLM_Flash.scr.inference import get_activation_model
# Register hook for complex model
model = LlamaModel(...)
hook = model.layers[0].register_forward_hook(
    get_activation_model('layer_0')
)
# Forward pass with tuple output
output, kv_cache = model(x)
# Access captured activations
data = activations['layer_0']
print(data['activation'].shape)

---

## print_info_model

`print_info_model(model, x)`

### Description

Prints comprehensive information about a model's layers, parameters, and
activations. Handles complex models with tuple outputs.

### Parameters

|Parameter|Type|Description|
|---|---|---|
|`model`|`nn.Module`|The neural network model to analyze|
|`x`|`torch.Tensor`|Input tensor for the forward pass|

### Returns

`torch.Tensor` - The output of the forward pass

### Sample Output
````
````

```
================================================================================
==
Input data: torch.Size([1, 10])
================================================================================
==
Layer: model.layers.0.self_attn.q_proj (Linear):
----------------------------------------
  weight: torch.Size([4096, 4096])
    Values: mean=0.000123, std=0.123456, range=[-0.543210, 0.567890]
  bias: torch.Size([4096])
    Values: mean=0.000001, std=0.000123, range=[-0.000456, 0.000789]
  Activation: torch.Size([1, 10, 4096]) (type: tensor)
    Statistics: mean=0.045678, std=0.234567, range=[-0.123456, 0.567890]
Final output: torch.Size([1, 10, 32000])
```

---

get_model_linear_activation

`get_model_linear_activation(name)`

Description

Creates a forward hook that captures activations only for Linear layers.

Parameters

Parameter	Type	Description
`name`	`str`	Name identifier for the layer being monitored

Returns

`function` - A hook function that captures Linear layer activations

Examples

python

```
from Jumping_LLM_Flash.scr.inference import get_model_linear_activation
linear_activations = {}
globals()['linear_activations'] = linear_activations
# Register hooks only for Linear layers
hooks = []
for name, module in model.named_modules():
```

```
    if isinstance(module, nn.Linear):
        hook = module.register_forward_hook(
            get_model_linear_activation(name)
        )
        hooks.append(hook)
# Forward pass
output = model(x)
# Access captured data
for name, data in linear_activations.items():
    print(f"{name}: weights shape {data['weights'].shape}")
# Clean up
for hook in hooks:
    hook.remove()
```

---

quantize_model_linear_layers

`quantize_model_linear_layers(model, dtype=torch.float32)`

Description

Quantizes only the Linear layers in a model using 8-bit quantization.

Parameters

Parameter	Type	Default	Description
`model`	`nn.Module`	-	The neural network model to quantize
`dtype`	`torch.dtype`	`torch.float32`	Data type for dequantized weights

Returns

`dict` - Quantization parameters

Examples

python

```
from Jumping_LLM_Flash.scr.inference import quantize_model_linear_layers
# Quantize LLaMA model
quant_params = quantize_model_linear_layers(model)
# Check quantization results
for name, params in quant_params.items():
    print(f"{name}: scale={params['scale']:.6f}, "
          f"zero_point={params['zero_point']:.2f}")
```

```
# Test quantized model
with torch.no_grad():
    output = model(x)
    print(f"Output shape: {output.shape}")
```

---

quantize_model_simple

`quantize_model_simple(model, x, vocab_size=32000)`

Description

Simplified model quantization process that combines analysis and quantization.

Parameters

Parameter	Type	Default	Description
`model`	`nn.Module`	-	Model to quantize
`x`	`torch.Tensor`	-	Test input tensor
`vocab_size`	`int`	`32000`	Vocabulary size for generation testing

Returns

`tuple` - (quantized_model, quant_params)

Examples

```
from Jumping_LLM_Flash.scr.inference import quantize_model_simple
# Load model
model = LlamaModel(vocab_size=32000, ...)
x = torch.randint(0, 32000, (1, 10))
# Quantize and verify
quantized_model, params = quantize_model_simple(model, x)
# Use quantized model
output = quantized_model.generate(x, max_new_tokens=10)
```

---

quantize_model_layer_weights

`quantize_model_layer_weights(layer)`

Description

Quantizes weights of a single Linear layer and returns detailed results.

Parameters

Parameter	Type	Description
`layer`	`nn.Linear`	The Linear layer to quantize

Returns

`dict` or `None` - Dictionary with quantization results

Examples

```
from Jumping_LLM_Flash.scr.inference import quantize_model_layer_weights
# Quantize a specific layer
layer = model.layers[0].self_attn.o_proj
result = quantize_model_layer_weights(layer)
if result:
    print(f"Quantization error: {result['error']:.6f}")
    print(f"Scale: {result['scale']:.6f}")
    # Update layer weights
    with torch.no_grad():
        layer.weight.data = result['dequantized']
```

---

quantize_language_model

`quantize_language_model(model, vocab_size=32000, test_seq_len=10)`

Description

Main function for quantizing LLaMA-style language models.

Parameters

Parameter	Type	Default	Description
`model`	`nn.Module`	-	LLaMA-style language model
`vocab_size`	`int`	`32000`	Vocabulary size
`test_seq_len`	`int`	`10`	Length of test sequence

Returns

`nn.Module` - Quantized model

Examples

python

```
from Jumping_LLM_Flash.scr.inference import quantize_language_model
# Load and quantize LLaMA model
model = LlamaModel.from_pretrained('path/to/model')
# Quantize with default parameters
quantized_model = quantize_language_model(
    model,
    vocab_size=32000,
    test_seq_len=10
)
# Test quantized model
test_input = torch.randint(0, 32000, (1, 5))
output = quantized_model.generate(test_input, max_new_tokens=10)
```

Complete Workflow Example

```
from Jumping_LLM_Flash.scr.inference import *
import torch
import torch.nn as nn
# 1. Create or load model
model = nn.Sequential(
    nn.Linear(10, 20),
    nn.ReLU(),
    nn.Linear(20, 10),
    nn.ReLU(),
    nn.Linear(10, 2)
)
# 2. Create test input
x = torch.randn(2, 10)
# 3. Analyze model
print("MODEL ANALYSIS")
print("=" * 50)
output = print_info_model(model, x)
# 4. Quantize linear layers
```

```
print("\nQUANTIZATION")
print("=" * 50)
quant_params = quantize_model_linear_layers(model)
# 5. Verify quantization
print("\nVERIFICATION")
print("=" * 50)
with torch.no_grad():
    quantized_output = model(x)

    # Calculate error
    error = torch.abs(output - quantized_output)
    print(f"Mean error: {error.mean().item():.6f}")
    print(f"Max error: {error.max().item():.6f}")
# 6. Print summary
print("\nQUANTIZATION SUMMARY")
print("=" * 50)
for name, params in quant_params.items():
    print(f"{name}:")
    print(f"  Shape: {params['original_shape']}")
    print(f"  Scale: {params['scale'].item():.6f}")
    print(f"  Zero point: {params['zero_point'].item():.2f}")
```

Quantization Parameters

Explanation of Quantization

Parameter	Description
Scale	Determines the step size: `scale = (max - min) / 255`
Zero Point	Maps value 0 in float to quantized integer: `zero_point = round(-min / scale)`
Quantized Range	[0, 255] for 8-bit quantization
Dequantization	`x_float = (x_quant - zero_point) * scale`

Error Metrics

Metric	Description
Mean Error	Average absolute difference between original and quantized weights
Max Error	Maximum absolute difference
Relative Error	`error / range` - percentage of precision loss

```
## Use Cases

### Model Compression

- Reduce memory footprint by storing weights as uint8

- Achieve ~4x compression for Linear layers


### Inference Acceleration

- Quantized operations are faster on supported hardware

- Reduce memory bandwidth requirements


### Edge Deployment

- Deploy models to resource-constrained devices

- Maintain reasonable accuracy while reducing size


### Model Debugging

- Inspect layer activations and statistics

- Identify potential issues in model architecture


---
```